Автоматизація збору: API / скрапінг / пайплайни та гігієна даних

Заняття №9

AI для OSINT і розвідки

r_d

ПЛАН ЗАНЯТТЯ



- Навіщо автоматизація: три способи збору і керовані платформи
- Анатомія пайплайну, гігієна даних, FollowTheMoney
- Демо 1: медіа-моніторинг → Obsidian; словник n8n ↔ Python
- Демо 2: скрапінг без API — портал ГУР, ScrapingBee, архів сайту
- Демо 3: збагачення об ′єкта через OpenSanctions → звіт
- Моніторинг, етика і OPSEC, порівняння інструментів

НАВІЩО АВТОМАТИЗАЦІЯ

Вручну

- 20 джерел перевірити руками — близько двох годин
- Щоразу трохи по-іншому: інші запити, інший порядок
- Без дат збору і без сирих копій — звіт не відтворити
- Дублі з різних джерел рахуються як «підтвердження»

Автоматично, але з гігієною

- Той самий збір — близько трьох хвилин, і щоразу однаково
- Кожен запис несе: звідки, коли, яким збирачем, за яким запитом
- Сирий HTML чи JSON лежить поруч із витягом — є куди повернутись
- Дублі зникають за стабільним id, а не «на око»

Мета автоматизації — не швидкість, а відтворюваний і аудитований збір. Аналіз і судження лишаються за людиною.

API VS СКРАПІНГ

- **API-запити:** дають структуровані дані (JSON), офіційні інтерфейси (Telegram API, VK API тощо). Потрібні ключі/токени та реєстрація додатку. Переваги – стабільність формату, недолік – rate-limit та геоблокування
- **Веб-скрапінг (requests+BS4):** гнучкий доступ до публічних HTML-сторінок без API. Необхідно парсити відповіді вручну (BeautifulSoup). Мінуси – зміна розмітки сайту, блокування (User-Agent, проксі).
- **Комбінація:** Використовуємо обидва підходи: напр., робимо GET до державного API реєстру і парсимо новинні сайти через requests+BeautifulSoup. Для телеграму – Telethon (API Telegram).

Висновок: три способи — API, скрапінг, RSS — плюс керовані платформи. Сьогодні показую їх у n8n, бо там кожен крок видно; у Python це requests, BeautifulSoup і feedparser — ті самі три гілки.

ТРИ СПОСОБИ + КЕРОВАНІ ПЛАТФОРМИ

API

- JSON за документацією, стабільний формат
- Ключ, реєстрація, ліміти, іноді геоблок
- Приклади: OpenSanctions, Tavily, Shodan, Jina Reader

n8n · Python: HTTP Request · requests

Скрапінг

- HTML публічної сторінки → CSS-селектори
- Ламається зі зміною розмітки; можливі 403 і CAPTCHA
- Приклади: t.me/s/<канал>, портал ГУР «Війна і санкції», сайти компаній

n8n · Python: HTML Extract + Code · BeautifulSoup

RSS / фіди

- Найдешевше, якщо фід є: без ключів і парсингу
- Лише свіжі записи, без історії
- Приклади: Укрінформ, УП, Суспільне; RSSHub як міст для Telegram

n8n · Python: RSS Read · feedparser

Керовані платформи ScrapingBee, Apify, Bright Data: браузер, проксі, знімки за кредити. Умови сайту лишаються вашим ризиком, вендор бачить ваші цілі. ScrapingBee — у демо 2, на легальній цілі.

АНАТОМІЯ ПАЙПЛАЙНУ: ЧОТИРИ ФАЗИ

1. Збір	RSS Read · HTTP Request + HTML · HTTP Request до API (Python: feedparser, requests, BeautifulSoup). Три формати на вході, Merge на виході.
2. Дедуплікація	Code: канонічний запис і стабільний id (хеш платформи + URL) → Remove Duplicates у прогоні та між прогонами (Python: set, SQLite).
3. Збагачення	LLM-вузол: резюме і сутності; API-збагачувачі: OpenSanctions, crt.sh, ip-api, Wayback. Дані в тегах, не інструкції.
4. Зберігання	Convert to File → Write File (Python: pathlib) у папку Obsidian vault: нотатка .md із frontmatter поруч із сирим .json.
Моніторинг	Той самий пайплайн + Schedule Trigger або cron + дедуплікація між прогонами. Історія виконань або логи = аудит-лог.

Кожна фаза — окремий вузол або функція: видно, що увійшло і що вийшло. Це і є «перевірити кожен крок» із Заняття 2.

ГІГІЄНА ДАНИХ: КАНОНІЧНИЙ ЗАПИС

П’ять принципів

- Нормалізація: UTF-8, дати в ISO 8601 і UTC, однакові ключі полів
- Дедуплікація за стабільним id, а не за схожістю заголовків
- Атрибуція: source_url, collector, query, collected_at у кожному записі
- Версіювання: сирий HTML або JSON зберігається поруч із витягом
- Аудит-лог: хто, коли, яким воркфлоу — історія виконань

```
id: 9f2c…e1   (хеш platform + url)
platform: telegram | rss | web
collector: HTTP Request + HTML
source_url: https://t.me/suspilnenews/78235
title: …
published_at: 2026-10-26T11:34:08Z
collected_at: 2026-10-26T18:02:10Z
query: медіа-моніторинг …
raw: raw/2026-10-26_telegram_9f2c….json
```

Якщо звіт не можна відтворити за цими полями — це не збір, а враження.

FOLLOWTHEMONEY: МОВА СУТНОСТЕЙ

Що це

- Сутність = {id, schema, properties}: одна модель для людей, компаній, адрес і зв ′язків
- Схеми: Person, Company, LegalEntity, Address, Sanction, Directorship, Ownership, Family
- Зв ′язок — теж сутність: Directorship має director, organization, role, startDate
- Той самий формат в Aleph (OCCRP), OpenSanctions, yente; утиліта ftm експортує в Neo4j і Gephi — міст до Заняття 6

```
{
  "id": "NK-jLcvcAFxBd2aYNeqUvit3a",
  "schema": "Company",
  "properties": {
    "name": ["Rosoboronexport JSC"],
    "innCode": ["7718852163"],
    "website": ["https://roe.ru/"],
    "topics": ["sanction",
"export.control"],
    "programId": ["EU-UKR", "UA-SA1644"]
  }
}
```

Одна функція перекладає нормалізований запис у FtM, а відповідь OpenSanctions уже приходить у FtM. Одна мова — один граф.

ДЖЕРЕЛА-API ДЛЯ ЗБАГАЧЕННЯ

Санкції та PEP

- OpenSanctions: OFAC, ЄС, Британія, РНБО, ГУР «Війна і санкції», RuPEP, список ФБК — одна точка
- Ключ безкоштовний для ЗМІ, НУО, академії, держустанов України; інакше ≈ 0,10 € за виклик
- Bulk-дані безкоштовно (CC BY-NC); yente — свій сервер

Сьогодні: у демо 3

Реєстри компаній

- OpenCorporates: ключ за заявкою з доказом публічної користі, ліміти ≈ 500/міс
- North Data: 27 країн без РФ, API за договором від 6 місяців, скрапінг сайту заборонено
- Україна: ЄДР через data.gov.ua; YouControl, Opendatabot — платні API

Сьогодні: лише огляд

Витоки і люди

- LeakCheck, HIBP, BreachDirectory: email, паролі, телефони
- Персональні дані: лише за мандатом, з ізольованого контуру
- Не з лекції і не з робочого акаунта

Сьогодні: не показуємо

Без ключа і без реєстрації: crt.sh (сертифікати → піддомени), ip-api (geo, ASN), Wayback (архів), Wikidata. Усі — у демо 3.

ДЕМО 1 · МЕДІА-МОНІТОРИНГ У N8N

Що показую

1. Налаштування: шлях до vault, RSS Укрінформу, канал suspilnenews, запит для Tavily
2. Три гілки збору: RSS Read · HTTP + HTML на t.me/s · Tavily API
3. Merge → Нормалізація → Дедуплікація → Ліміт 10
4. Резюме LLM (Gemini) з правилом безпеки із Заняття 2
5. Запис у vault → Obsidian: inbox і Dataview-таблиця

На що дивитись

- Три різні формати на вході — один запис на виході
- Поля атрибуції в кожному записі; id — хеш
- Сирий JSON лежить поруч у raw/
- Розклад 30 хв уже стоїть; «між прогонами» вимкнено для демо
- Клік на будь-який вузол — вхід і вихід фази

Вузли з коробки плюс три Code-вузли, і код у них — нормалізація, а не збір. У Python це той самий скрипт на ~120 рядків.

ЩО ЛЕЖИТЬ У VAULT ПІСЛЯ ПРОГОНУ

```
OSINT-vault/
  inbox/     2026-10-26_telegram_9f2c….md
             2026-10-26_rss_41aa….md
  raw/       2026-10-26_telegram_9f2c….json
  entities/  Rosoboronexport.md
             Sergey_Sergeevich_Chemezov.md
  reports/   report_Rosoboronexport.md
             sources_Rosoboronexport.md
  attachments/ gur_Rosoboronexport.png
  Монітор.md   ← Dataview-таблиця
```

Dataview: таблиця без бази даних

```dataview
TABLE platform, source_name, published_at
FROM "inbox"
SORT published_at DESC
```

- Frontmatter нотатки = колонки таблиці
- Wikilinks між сутностями = граф Obsidian
- Сирі файли поруч — до джерела за один клік

Obsidian тут — аналітичний шар: таблиці, граф і нотатки аналітика над тими самими файлами, що пише пайплайн.

СЛОВНИК: ВУЗОЛ N8N ⇔ PYTHON

Збір і розбір

HTTP Request → requests.get(url, headers=UA, timeout=30)

HTML Extract → BeautifulSoup(html).select('.card')

RSS Read → feedparser.parse(url).entries

Code → звичайна функція normalize(record)

LLM-вузол → SDK моделі: system + user, відповідь JSON

Convert + Write File → Path(vault, 'inbox', name).write_text(md)

Пайплайн і режим

Merge → list.extend або asyncio.gather для гілок

Remove Duplicates → set(id) у прогоні; таблиця seen у SQLite між прогонами

Retry on Fail → tenacity: @retry(wait=wait_exponential(3, 48))

Batching → time.sleep(2) між запитами; семафор для паралелі

Schedule Trigger → cron, APScheduler або systemd timer

Executions → logging у файл + збережені raw/*.json

Воркфлоу — це той самий скрипт, у якому кожну функцію видно як вузол. Схема і гігієна однакові; інструмент обирайте свій.

ДЕМО 2 · СКРАПІНГ БЕЗ API

Що показую

1. Портал ГУР «Війна і санкції»: API немає, robots дозволяє → HTTP Request із чесним User-Agent + HTML Extract: 12 компаній «Ростеху»

2. Картка об ′єкта через ScrapingBee: extract_rules → JSON без парсера; screenshot → знімок-доказ у vault

3. Сайт об ′єкта roe.ru з-за кордону: 503/404 — не обходимо, беремо архівну копію Wayback: прес-релізи з датами

4. Merge → нотатка «Первинні джерела» + 13 нотаток сутностей → граф в Obsidian

На що дивитись

- ІПН 7718852163 і ОГРН з ГУР збігаються з OpenSanctions — перевірка незалежним джерелом
- 8 юрисдикцій, кожна з датою і документом — атрибуція з первинного джерела
- Скриншот — доказ: сторінка може змінитись або зникнути
- Відмова сайту — теж дані: фіксуємо статус і дату, не обходимо
- Кредити ScrapingBee: 1 без JS · 5 із JS · 10–25 через проксі

Платформа потрібна не для обходу, а там, де без браузера сторінка порожня або потрібен знімок. Решту робить звичайний запит.

ТИПОВІ ПРОБЛЕМИ ТА РІШЕННЯ

HTTP 429 (Too Many Requests): Виникає при частих запитах. *Рішення:* Додавати паузи між запитами,, використовувати механізм exponential backoff або розподілити запити через проксі.

Блокування API: Сайт відповідає 403 або CAPTCHA. *Рішення:* офіційний API або RSS, якщо є; архівна копія; керована платформа з оглядкою на ToS і OPSEC; для Telegram — публічне прев ′ ю t.me/s з паузами.

Некоректний ввід: Перевіряйте назву, домен, ідентифікатори (ІПН, ОГРН). Виконуйте **input validation** у функції валідації до першого запиту (Code-вузол або функція).

Неповні дані: Якщо збагачувач не відповів — пайплайн не вигадує, а позначає «не встановлено» (continue on error + розділ «прогалини» у звіті). Джерела додаємо як окремі гілки.

Гарні практики: Розбивайте великі запити на менші, індексуйте та логируйте дані для повторної перевірки.

ТИПОВІ ПРОБЛЕМИ ТА РІШЕННЯ

- **Стандартизація:** Нормалізуйте дані (дата, час, формати телефонів/ФІО) при об’єднанні із різних джерел.
- **Перевірка з незалежними джерелами:** Для критичної інформації – знаходимо підтвердження у різних джерелах. Наприклад, адреса або ІПН повинні співпадати в реєстрі та на сайті.
- **Видалення дублікатів:** Якщо одна й та сама інформація прийшла з різних місць, фільтруйте дублі (наприклад, використайте хеш composite key: hash(username+platform)).
- **Формат виводу:** Генеруйте структуровані формати (JSON, CSV) з уніфікованими ключами для подальшої обробки.
- **Збереження raw-даних:** Логіруйте сирі відповіді API та HTML для аудиту та повернення до джерела.

ПЕРЕРВА

5 хвилин

Після перерви:

- Демо 3: збагачення об ′єкта через OpenSanctions
- FollowTheMoney і граф у Obsidian
- Звіт і моніторинг за розкладом
- Етика, легальність, OPSEC
- Порівняння інструментів і висновки

ДЕМО 3 · ЗБАГАЧЕННЯ ОБ'ЄКТА

Що показую

1. Налаштування: об ′єкт Rosoboronexport, домен roe.ru; Tavily — через credential n8n, ключ OpenSanctions у полі
2. OpenSanctions search → обрати сутність → entity з nested-зв ′язками
3. Паралельно: crt.sh, ip-api, Wayback, Tavily-новини
4. Merge → Збірка досьє у FtM → LLM → JSON із ризиком і прогалинами
5. Звіт і нотатки сутностей у vault → граф в Obsidian

На що дивитись

- Один об ′єкт, п ′ять джерел, один Merge — без черг і воркерів
- 32 санкційні записи, директорство і власність — з одного виклику
- Пов ′язані особи стають окремими нотатками з wikilinks
- LLM відповідає JSON і прив ′язує кожен red flag до поля досьє
- Розділ «прогалини» — обов ′язковий

Схема класична: збір з реєстрів, новини, AI-аналіз, звіт. У Python це скрипт, у n8n — вузли, які видно на кожному кроці.

OPENSANCTIONS: ТРИ МЕТОДИ API

/search

- Швидкий пошук за рядком: q=Rosoboronexport
- Фільтри: schema, countries, topics, datasets
- Повертає кандидатів із caption, topics, датасетами

Коли: перший крок, щоб знайти id

/entities/{id}?nested=true

- Повна FtM-сутність: аліаси, ІПН/ОГРН, адреси, сайт
- Санкції: орган, програма, дата кожного запису
- Зв ′язки: directorship, ownership, family, associates, succession
- first_seen, last_seen, datasets — готова атрибуція

Коли: досьє і граф

/match

- Структуроване зіставлення: schema + name + birthDate + nationality
- Кандидати зі score і прапорцем match
- Chemezov 1.0 · Shemetov 0.78 · Cherezov 0.75

Коли: перевірка «чи той це»

Score — не вердикт: 0.78 без дати народження — кандидат на перевірку, не факт. Поріг і правило вирішує аналітик.

AI-АНАЛІЗ: ФОРМАТ ВІДПОВІДІ



Приклад виходу (вигаданий):

```
{
  "overall_risk": "Низький",
  "status_changes": ["нових санкційних
записів за рік немає"],
  "red_flags": ["одноосібне
керівництво без співзасновників"],
  "recommendation": "Продовжити
взаємодію з моніторингом",
  "next_steps": ["перевірити фінансову
звітність за рік"]
}
```

ЗВІТ У OBSIDIAN: ЩО МАЄ БУТИ ВСЕРЕДИНІ

1. Об ′єкт	Назва, домен, ідентифікатори (ІПН, ОГРН, OpenSanctions id), дата збору, воркфлоу або скрипт, яким зібрано.
2. Джерела	Кожне джерело з датою: first_seen і last_seen у OpenSanctions, published_at новин, timestamp Wayback, знімок сторінки.
3. Сутності	FtM-сутності і зв ′язки; особи — окремі нотатки з wikilinks на компанію і на звіт.
4. AI-аналіз	JSON: ризик, впевненість, red flags із полем-джерелом, рекомендація, наступні кроки.
5. Прогалини	Чого не вдалося перевірити і де: crt.sh не відповів, реєстр платний, сайт об ′єкта недоступний з-за кордону.

Правило з Заняття 3: висновок відтворюється іншою людиною за вашими кроками. Тут кроки — вузли або функції, а файли — докази.

МОНІТОРИНГ ЗА РОЗКЛАДОМ

Як перетворити збір на моніторинг

- Запуск за розкладом: cron, APScheduler або Schedule Trigger — кожні 30 хв, щогодини
- Пам ′ять бачених id між прогонами: таблиця seen у SQLite або вузол Remove Duplicates
- Нове = те, що пройшло дедуплікацію; зникле — порівняння зі snapshot
- Сповіщення: нотатка в inbox, email, Telegram-бот

Що вважати сигналом

- Новий матеріал за ключовими словами з Налаштувань
- Новий санкційний запис або новий зв ′язок у сутності (last_change)
- Новий піддомен у crt.sh, зміна відповіді Wayback, зміна картки ГУР
- Сплеск: більше N записів за годину з одного джерела

Аудит-лог — не окремий проєкт: логи скрипта плюс raw/*.json або історія виконань n8n. Головне — щоб він був і його не чистили.

ЕТИКА, ЛЕГАЛЬНІСТЬ, OPSEC

Дозволено

- Публічні дані, API за умовами провайдера
- RSS і сторінки, які сайт сам віддає; архівні копії
- robots.txt і rate limits — поважаємо
- Атрибуція джерел у звіті

Сіра зона

- Скрапінг сайтів, чиї ToS це забороняють
- Обхід блокувань через проксі-мережі і stealth-режими
- Персональні дані без правової підстави: GDPR і CCPA мають реальні штрафи
- Для комерційного збору — юрист

Заборонено

- Злам, обхід автентифікації і CAPTCHA
- Витоки паролів для доступу або тиску
- Масовий збір персональних даних громадян ЄС без підстави
- Чутливі цілі з робочого або особистого акаунта

OPSEC для цього курсу: окремий контур (VM або профіль), окремі ключі й акаунти. Вендор платформи бачить усе, що ви в неї вводите.

ПОРІВНЯННЯ ІНСТРУМЕНТІВ OSINT

Інструмент	Опис	Вхідні дані	Вихід	Обмеження
Low-code (n8n, Make) + API	Low-code пайплайн: HTTP, Code, LLM-вузли	Назва, домен, URL, фід	Нотатки і звіт в Obsidian	Ключі API окремо; ліміти джерел
Quick OSINT	SaaS (акцент на VK)	Логін VK або телефон	Дані VK профілів та email	Платний RU-сервіс: не з робочих акаунтів
Gaze Bot	Telegram-бот (акаунти)	Нікнейм Telegram	Інфо про акаунт/канал Telegram	Платний RU-бот, логує запити: OPSEC-ризик
Python + requests/BS4	Власний скрапер (гібра)	URL або API запит	HTML/JSON для подальшого парсингу	Потрібно писати код; самостійна обробка даних
Telethon (Python)	Бібліотека для Telegram API	api_id, api_hash, запит	Telegram-повідомлення	Потрібен реєстраційний Telegram-акаунт
Ollama / OpenAI (LLM)	Локальний/хмарний LLM для аналізу текстів	Текстові дані, prompt	Структурований звіт (JSON)	Ліміти токенів, вартість API (якщо платно)

ВИСНОВКИ

Гібридний підхід: API, скрапінг і RSS живуть в одному пайплайні як окремі гілки; реєстри дають точні поля, новини й канали — контекст.

Контроль за даними: канонічний запис із атрибуцією, стабільний id, сирі відповіді поруч із витягом. Без цього збір не відтворити.

AI-аналіз: LLM працює лише над зібраним, відповідає JSON із прив ′язкою до полів досьє і обов ′язковим розділом «прогалини».

Відтворюваність: пайплайн — це код: скрипт у git або експорт воркфлоу в JSON, логи або історія виконань, vault як докази.

Практика: чотири фази однакові для Python і n8n; JSON воркфлоу з лекції — референс схеми, а не обов ′язковий інструмент.

Q&A

???

r_d

ЗАВЖДИ Є КУДИ ЗРОСТАТИ